

Document de synthèse de la PoC

Allocation de lits d'urgence

(MedHead)

Ce document synthétise les décisions prises dans le cadre du développement d'une preuve de concept pour le consortium de santé MedHead, composé de quatre organisations : Ursa Major Health, Jupiter Scheduling, Emergency Expert Systems et Schedule Shed dont les systèmes sont aujourd'hui mal intégrés et synchronisés manuellement, une situation incompatible avec l'urgence médicale, qui exige une réponse en temps réel. L'objectif de la PoC est d'orienter un patient en situation d'urgence vers l'hôpital compétent le plus proche, en tenant compte à la fois de la spécialité médicale requise, de la disponibilité d'un lit, et du temps de trajet réel plutôt que d'une distance à vol d'oiseau, qui ne reflète pas la réalité du terrain.

Les attentes du projet se déclinent en cinq étapes : la mise en place de l'environnement de travail, le développement du backend, celui du frontend, la mise en place d'un pipeline d'intégration et de livraison continues, et enfin la rédaction de la documentation et du reporting. Le présent document couvre, dans cet ordre, le périmètre retenu pour la PoC, sa conformité RGPD, les choix d'architecture, les critères de succès et leur méthode de démonstration, le planning, les décisions Git et CI/CD, la justification des choix technologiques, les enseignements tirés du projet, et les recommandations à suivre pour une mise en production.

1. Scope de la PoC

La PoC se limite à une API REST qui reçoit en entrée la localisation du patient et la spécialité médicale recherchée, et retourne en sortie l'hôpital le plus proche disposant à la fois d'un lit disponible et de la spécialité demandée. Elle publie par ailleurs un événement de réservation de lit, de manière asynchrone et découplée de la réponse HTTP synchrone. Un frontend minimaliste permet de sélectionner une spécialité et de saisir une localisation, en consommant cette API.

Plusieurs éléments sont explicitement exclus du périmètre : l'identification ou la modélisation du patient aucun nom, aucun identifiant persistant n'est stocké, le tri par symptômes, hors-projet selon le Document de définition de l'architecture, l'intégration réelle avec les systèmes tiers du consortium (Ursa Major, Jupiter, Schedule Shed), qui restent factices ou mockés, ainsi qu'un véritable bus d'événements de type Kafka ou RabbitMQ, remplacé par un adaptateur mock. Le recours à plusieurs fournisseurs de géolocalisation en cascade en cas de panne est documenté comme une recommandation pour la production, mais n'est pas codé dans la PoC.

La stack technique retenue repose sur un backend en Java et Spring Boot, prêt pour une intégration microservice, et un frontend en React. L'architecture métier suit un pattern hexagonal, en ports et adaptateurs, conforme au Principe B2 et à la couche « Services et capacités limités au domaine » du Document de définition de l'architecture.

2. Conformité RGPD

La conformité RGPD est démontrée par la conception plutôt que par des mesures ajoutées après coup. Le principe de minimisation des données (Art. 5) est respecté : aucune donnée patient non nécessaire au calcul n'est collectée, ni nom ni identifiant persistant. Le principe de limitation de la conservation (Art. 5.1.e) l'est également : la localisation n'est traitée qu'en mémoire, pour le seul temps du calcul de distance, et n'est jamais persistée en base.

Un point reste ouvert et documenté comme tel : la localisation est transmise à un fournisseur de géolocalisation tiers pour le calcul de la distance réelle. Ce fournisseur étant mocké dans la PoC, la question du respect du chapitre V du RGPD, relatif aux transferts hors Union européenne, n'est pas résolue à ce stade. En production, le choix du fournisseur devra intégrer un critère RGPD explicite conformité, localisation des traitements, garanties contractuelles si le fournisseur est situé hors UE un choix distinct de celui de l'hébergeur du service lui-même, développé en section 9.

3. Architecture: points clés

La publication de la réservation de lit repose sur un port `PublicateurEvenement`, respectant le pattern ports/adaptateurs. Un seul adaptateur est implémenté pour la PoC : un mock qui enregistre l'événement dans un log et une liste en mémoire, exploitable directement dans les tests. Un véritable adaptateur, de type Kafka ou RabbitMQ, n'est volontairement pas codé, ce qui reste cohérent avec le Principe C4 d'isolation des données et systèmes de production, et avec les simplifications suggérées par la Déclaration des travaux d'architecture.

Le calcul de distance doit refléter un temps de trajet réel, jamais une distance à vol d'oiseau en fonctionnement normal. Il repose sur un port `CalculateurDistance`, dont l'adaptateur réel appelle un service externe de géolocalisation. Cet appel est protégé par un circuit breaker, complété par un cache des positions des hôpitaux — des données stables, ce qui limite les appels externes. En cas de panne ou de latence, un mécanisme de repli bascule sur une distance à vol d'oiseau, avec un indicateur explicite de dégradation dans la réponse, par exemple un champ `precision: "estimee"`, afin de respecter le principe « d'abord ne pas nuire » et d'éviter qu'une donnée dégradée soit prise pour une donnée fiable. Le recours à plusieurs fournisseurs externes en cascade reste une recommandation documentée pour la production, non implémentée dans la PoC.

4. Critères de succès et méthode de démonstration

Cinq critères de succès ont été définis par les Exigences de la PoC. Le tableau suivant les récapitule et précise, pour chacun, s'il est démontrable par la PoC seule et selon quelle méthode.

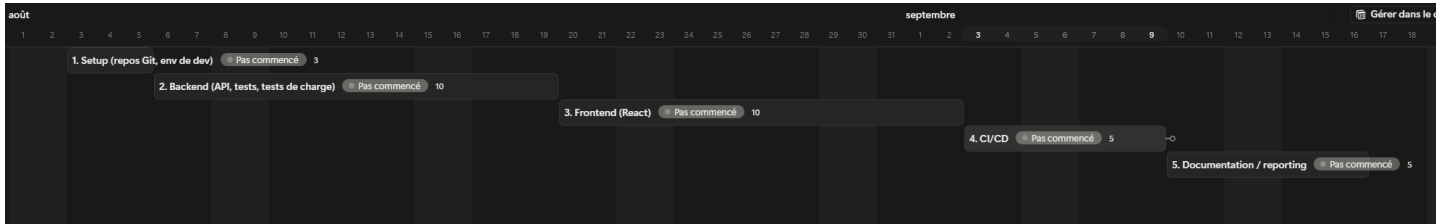
#	Critère (source : Exigences PoC)	Démontrable seule ?	Méthode de démonstration
1	> 90 % des cas acheminés vers l'hôpital compétent le plus proche	Oui	Jeu de données de test contrôlé (scénario Fred Brooks / Julia Crusher / Beverly Bashir) + tests automatisés
2	Temps moyen de traitement d'une urgence : 18,25 min → 12 min	Non, pas seule	KPI métier global incluant des étapes hors périmètre (transport, prise en charge sur site...). La PoC ne contribue qu'à la portion décision/orientation.
3	< 200 ms à 800 req/s par instance	Oui	Tests de charge JMeter sur l'endpoint principal
4	Respect des normes imposées	Oui	Deux piliers : RGPD (section 2) et architecture hexagonale / microservice-ready (section 3), démontrable via tests unitaires du domaine sans dépendance à Spring
5	Délai imparti respecté	Oui	Planning fixé et suivi (section 5)

Le test Jmeter annonce un temps de traitement de 3ms pour 95% des requête en pleine charge, se qui respecte amplement l'exigence de la poc

Requests	Executions			Response Times (ms)							Throughput
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s
Total	55899	0	0.00%	3.12	0	1320	1.00	2.00	3.00	150.98	800.81
POST /api/bed-allocations	55899	0	0.00%	3.12	0	1320	1.00	2.00	3.00	150.98	800.81

5. Planning

Aucune deadline n'étant imposée par l'énoncé, le délai a été fixé sur un total d'environ 33 jours ouvrés, soit 6,5 semaines. La mise en place de l'environnement de travail — repositories Git, environnement de développement — a été estimée à 3 jours. Le développement du backend, incluant l'API, les tests et les tests de charge, a été estimé à 10 jours, tout comme celui du frontend en React, ce dernier ayant été volontairement allongé compte tenu d'une stack moins maîtrisée. La mise en place du pipeline CI/CD et la rédaction de la documentation ont chacune été estimées à 5 jours.



6. Décisions Git et CI/CD

Le projet repose sur un seul repository de code, organisé en monorepo, conformément à l'attente de l'étape 1 : deux repositories au total pour le projet, l'un pour le code et l'autre pour l'architecture, et non deux repositories pour le seul code. Ce monorepo regroupe un dossier backend en Java et Spring Boot avec Maven, tests, Checkstyle et Spotless, un dossier frontend en React et Vite avec ESLint et Prettier, les workflows de CI, un modèle de pull request, un README et les fichiers .gitignore.

Le workflow Git retenu est GitHub Flow, sans branche develop. Cette décision a été challengée puis confirmée : une branche develop n'apporte pas de valeur ajoutée dans un contexte de développement solo, sur un projet court de 6,5 semaines, sans plusieurs versions à faire coexister. La branche main est protégée : toute fusion nécessite une pull request et une CI verte. Les messages de commit suivent la convention Conventional Commits, afin d'assurer la traçabilité attendue par le Principe B3.

La pipeline backend se lance dès qu'un fichier du dossier backend/ est modifié. Il commence par récupérer le code du dépôt, puis installe Java 25, la version utilisée par l'application. Il compile ensuite le projet et exécute l'ensemble des tests unitaires en calculant au passage la couverture de code, avant de générer le fichier .jar final de l'application. Ce jar, ainsi que les rapports de tests, sont conservés pendant sept jours sous forme d'artefacts téléchargeables, ce qui permet de les consulter plus tard si besoin, par exemple pour diagnostiquer un échec. Pour terminer, le pipeline envoie le code à SonarQube, qui analyse la qualité et la sécurité du code Java et remonte les problèmes détectés.

De la même façon, la pipeline frontend se déclenche dès qu'un fichier du dossier frontend/ change. Après avoir récupéré le code, il installe Node.js 22 et les dépendances du projet avec npm ci. Il vérifie ensuite que le code respecte les règles de style et de qualité définies pour le projet via le lint, puis lance les tests avec couverture de code. Une fois les tests validés, l'application est construite avec la commande de build, et le résultat est archivé pendant sept jours. Comme pour le backend, une analyse SonarQube est effectuée à la fin pour surveiller la qualité du code React.

La troisième pipeline nommé E2E est un peu différent : elle se déclenche dès qu'un changement touche le backend ou le frontend, car son rôle est de tester l'application dans son ensemble, comme le ferait un vrai utilisateur. Elle installe à la fois Java et Node.js, précompile rapidement le backend pour détecter très vite une erreur de compilation, puis installe les dépendances du frontend. Pour gagner du temps, les navigateurs utilisés par Playwright (l'outil de test E2E) sont mis en cache d'une exécution à l'autre, évitant de retélécharger Chromium à chaque fois. Le pipeline lance ensuite les scénarios de test qui simulent une navigation complète dans l'application, du frontend jusqu'au backend. Si un test échoue, un rapport Playwright détaillé est automatiquement conservé pendant sept jours pour permettre d'analyser précisément ce qui a posé problème.

7. Justification des technologies

Le backend s'appuie sur Java et Spring Boot. Le typage statique du langage, vérifié dès la compilation — typage des variables, résolution des noms, respect du contrat des interfaces, gestion des exceptions vérifiées — élimine une classe entière de bugs avant même l'exécution, un atout dans un contexte médical où la fiabilité prime. Spring Boot, framework mature porté par une large communauté, offre un écosystème particulièrement adapté à une architecture microservices ou orientée services, cohérent avec l'exigence que la PoC puisse s'inscrire dans une telle architecture à terme.

Le frontend a été développé en React, parmi les frameworks JavaScript ou TypeScript courants du marché proposé par le cahier des charges. Ce choix répond à une logique d'évolutivité : la PoC peut servir de point de départ à l'évolution du produit, voire être intégrée comme composant réutilisable dans une autre interface, dans une logique de micro-frontend. React permet également de répondre à l'exigence d'un visuel dynamique, sans nécessiter un temps de développement disproportionné par rapport au périmètre restreint de la PoC.

La documentation de l'API repose enfin sur springdoc-openapi, qui génère automatiquement, à partir des annotations Spring, la spécification OpenAPI de l'endpoint principal, consultable via une interface Swagger et un point d'accès JSON dédié. Cette spécification, validée en amont, constituerait pour une équipe de développement un contrat d'API permettant au front-end et au back-end de démarrer en parallèle, sans attendre l'implémentation complète l'un de l'autre.

8. Enseignements de la PoC

Architecture hexagonale : un compromis qui a payé

La mise en œuvre de ports et d'adaptateurs a ajouté du volume de code et de la complexité initiale, en particulier sans expérience préalable du pattern. Ce surcoût a cependant été compensé par une charge de travail finalement inférieure à l'estimation, grâce à une portée limitée. Un seul connecteur externe réel à mocker et à une documentation d'architecture claire fournie en amont. Le pattern facilite par ailleurs la reprise du code par une équipe de développement, chaque partie pouvant être modifiée isolément sans casser le reste.

Limites du scope : la gestion des cas limites

La PoC ne définit pas de comportement explicite pour certains cas limites, comme l'absence d'hôpital compétent disposant d'un lit disponible, ou une spécialité inconnue de la liste : le comportement actuel y est indéfini. Ce choix reste cohérent avec le principe d'une PoC, qui est de ne pas s'éparpiller sur des cas hors du cœur de la démonstration, mais ne serait pas acceptable en production pour un système médical. Plutôt que d'anticiper seul tous les cas limites possibles, il serait préférable de mettre en place un processus de remontée des erreurs et des cas non gérés rencontrés, par exemple dans Notion, de les regrouper par fréquence et par criticité, et de les arbitrer avec le client lors des points de suivi, plutôt que d'en deviner seul les priorités.

Pilotage d'équipe : le contrat d'interface avant le développement parallèle

Le fait de développer seul a imposé de jouer successivement les rôles de développeur back-end et front-end. Si le projet avait été confié à une équipe, le contrat d'API, la spécification OpenAPI générée par springdoc-openapi aurait dû être défini et validé avant le début du développement, pour permettre aux deux équipes de travailler en parallèle sans se bloquer mutuellement. Un rituel quotidien pour remonter les blocages et les besoins au fil de l'eau, complété par un point hebdomadaire de vision d'ensemble, aurait structuré le pilotage de cette collaboration.

9. Recommandations pour la production

Plusieurs points, documentés ici mais non implémentés dans la PoC, restent à traiter avant une mise en production. Le recours à plusieurs fournisseurs de géolocalisation en cascade renforcerait la résilience du service au-delà du simple repli sur une distance à vol d'oiseau. Le choix du fournisseur de géolocalisation tiers devra intégrer un critère RGPD explicite pour la conformité, localisation des traitements, garanties contractuelles si le fournisseur est situé hors UE, un choix distinct de celui de l'hébergeur du service lui-même. Enfin, la mise en place d'un processus de remontée et d'arbitrage des cas limites non gérés par la PoC permettrait de prioriser les évolutions avec le client plutôt que de les anticiper seul.